

ARTIFICIAL INTELLIGENCE

Lab Handbook

Annexure: Lab Work — 16 Practical Sessions

Course Code:	[Insert Course Code]
Instructor:	[Insert Instructor Name]
Semester:	[Insert Semester / Year]
Lab Platform:	Python 3.10+ (NumPy, Pandas, Matplotlib, scikit-learn, TensorFlow/Keras)

Table of Contents

General Lab Guidelines	3
Lab 1: Introduction to Python for AI — Setup and Basic Data Handling	4
Lab 2: Implement a Simple Rational Agent in a Defined Environment	6
Lab 3: Implement & Visualize BFS/DFS for a Maze Pathfinding Problem	8
Lab 4: Compare Performance of Uninformed vs. Informed Search	10
Lab 5: Implement A* Search to Solve the 8-Puzzle Problem	12
Lab 6: Implement Hill Climbing; Analyze Local Optima	14
Lab 7: Develop a Tic-Tac-Toe Game with the Min-Max Algorithm	16
Lab 8: Enhance the Game with Alpha-Beta Pruning	18
Lab 9: Build a Small Rule-Based Expert System	20
Lab 10: Build a Decision Tree Classifier From Scratch	22
Lab 11: Implement a Naïve Bayes Classifier for Spam Detection	24
Lab 12: Compare Naïve Bayes and SVM with scikit-learn	26
Lab 13: Build a Feedforward Network for Digit Classification	28
Lab 14: Implement an MLP From Scratch to Solve XOR	30
Lab 15: Implement a Genetic Algorithm (Eight Queens)	32
Lab 16: Small-Scale AI Application — Pre-Trained CNN Classifier	34

General Lab Guidelines

This handbook accompanies the sixteen practical lab sessions of the Artificial Intelligence course. Each lab builds directly on the corresponding week's lecture material, moving from foundational Python and agent design, through classical search and adversarial games, into classical machine learning, and finally neural networks and a capstone AI application.

Environment Setup

All labs use Python 3.10 or later. Install the required packages once at the start of the semester:

```
pip install numpy pandas matplotlib scikit-learn tensorflow
```

Labs 13 and 16 require TensorFlow (or, if the instructor prefers, an equivalent PyTorch installation) and a working internet connection to download pre-trained weights and the MNIST dataset on first run.

How Each Lab Is Structured

- Objective — what the lab is meant to teach.
- Background — the theoretical concept the lab operationalizes, tied to the corresponding lecture week.
- Lab Task — a numbered, step-by-step specification of what to build.
- Code Example — a working Python reference implementation, provided as a scaffold and study aid, not a substitute for writing and understanding the code yourself.
- Expected Output / Discussion — what a correct run should produce, and what it demonstrates.
- Learning Outcomes — the specific skills assessed for that lab.

Submission and Academic Integrity

Unless otherwise instructed, submit your own working code (not copied verbatim from this handbook) along with a short write-up of your results, any deviations from the expected output, and answers to the discussion prompts. Code adapted from the examples in this handbook should demonstrate independent understanding — be prepared to explain any line of your submission if asked.

Lab 1: Introduction to Python for AI — Setup and Basic Data Handling

Objective

Set up a working Python environment for AI development and practice fundamental data-handling operations used throughout the course (lists, dictionaries, NumPy arrays, and Pandas DataFrames).

Background

Virtually every AI technique in this course — from search to neural networks — is implemented in Python, relying heavily on NumPy for numerical computation and Pandas for tabular data manipulation. Before any AI algorithm can be built, students must be comfortable representing state (lists, dicts, tuples), performing vectorized numeric operations (NumPy arrays), and loading/cleaning datasets (Pandas DataFrames). This lab establishes that foundation.

Lab Task

1. Install Python 3.10+, then install the required packages: `pip install numpy pandas matplotlib scikit-learn`.
2. Create basic Python data structures (lists, tuples, dictionaries) to represent a simple 'agent state' (e.g., position, energy level, sensor readings).
3. Use NumPy to create and manipulate arrays: perform element-wise arithmetic, reshaping, and basic statistics (mean, std, max).
4. Load a small CSV dataset with Pandas, inspect it (head, describe, info), handle a missing value, and filter rows based on a condition.
5. Plot a simple chart (e.g., histogram of a column) using Matplotlib to practice basic visualization.

Code Example (Python)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Basic Python data structures representing an agent's state
agent_state = {
    'position': (2, 3),
    'energy': 87,
    'sensors': ['bump', 'clear', 'clear', 'bump']
}
print('Agent state:', agent_state)

# 2. NumPy array operations
readings = np.array([12.5, 9.3, 15.7, 20.1, 8.4])
print('Mean reading:', readings.mean())
print('Std deviation:', readings.std())
normalized = (readings - readings.min()) / (readings.max() - readings.min())
print('Normalized readings:', normalized)

# 3. Pandas DataFrame handling
df = pd.DataFrame({
    'trial': [1, 2, 3, 4, 5],
    'success': [1, 0, 1, 1, None],
    'time_sec': [3.2, 5.1, 2.8, 4.0, 3.9]
})
print(df.describe())
df['success'] = df['success'].fillna(0)          # handle missing value
fast_trials = df[df['time_sec'] < 4.0]          # filter rows
print('Fast trials:')
print(fast_trials)
```

```
# 4. Basic visualization
plt.hist(readings, bins=5)
plt.title('Sensor Reading Distribution')
plt.xlabel('Reading value')
plt.ylabel('Frequency')
plt.savefig('lab1_histogram.png')
print('Histogram saved to lab1_histogram.png')
```

Expected Output / Discussion

The script prints the agent's state dictionary, summary statistics of the NumPy array, the DataFrame's descriptive statistics, the filtered fast-trial rows, and saves a histogram image confirming the plotting pipeline works.

Learning Outcomes

- Comfortable creating and manipulating core Python data structures.
- Able to perform vectorized numeric computation with NumPy.
- Able to load, clean, and filter tabular data with Pandas.
- Able to produce a basic Matplotlib visualization.

Lab 2: Implement a Simple Rational Agent in a Defined Environment

Objective

Design and implement a rational agent using the PEAS framework, operating in a simple, fully-observable, deterministic environment (a 2-cell vacuum world), and connect this to the agent taxonomy from Week 1 (simple reflex, model-based, goal-based, utility-based).

Background

A rational agent selects the action that maximizes its expected performance measure given its percept sequence. The classic teaching example is Russell & Norvig's vacuum-cleaner world: two locations (A and B), each either Clean or Dirty. This lab implements progressively more sophisticated agents operating in that environment, mirroring the agent hierarchy covered in lecture.

Lab Task

1. Model the environment: two locations ('A', 'B'), each with a state of 'Clean' or 'Dirty', and an agent location.
2. Implement a Simple Reflex Agent: it reacts only to the current percept (location, status) using condition-action rules.
3. Implement a Model-Based Agent: it remembers which locations it has already cleaned, avoiding redundant actions.
4. Run both agents for a fixed number of steps on a randomized initial environment and record the performance measure (e.g., +1 per clean location per time step, -1 per movement).
5. Compare the two agents' cumulative performance scores and discuss why the model-based agent may outperform the simple reflex agent.

Code Example (Python)

```
import random

class VacuumEnvironment:
    def __init__(self):
        self.status = {'A': random.choice(['Clean', 'Dirty']),
                       'B': random.choice(['Clean', 'Dirty'])}
        self.agent_location = random.choice(['A', 'B'])
        self.performance = 0

    def percept(self):
        return (self.agent_location, self.status[self.agent_location])

    def execute(self, action):
        if action == 'Suck':
            if self.status[self.agent_location] == 'Dirty':
                self.status[self.agent_location] = 'Clean'
        elif action == 'Right':
            self.agent_location = 'B'
            self.performance -= 1
        elif action == 'Left':
            self.agent_location = 'A'
            self.performance -= 1
        for loc in self.status:
            if self.status[loc] == 'Clean':
                self.performance += 1

def simple_reflex_agent(percept):
    location, status = percept
    if status == 'Dirty':
        return 'Suck'
    return 'Right' if location == 'A' else 'Left'
```

```

class ModelBasedAgent:
    def __init__(self):
        self.model = {'A': None, 'B': None}    # remembered status

    def act(self, percept):
        location, status = percept
        self.model[location] = status
        if status == 'Dirty':
            return 'Suck'
        if self.model['A'] == 'Clean' and self.model['B'] == 'Clean':
            return 'NoOp'
        return 'Right' if location == 'A' else 'Left'

def run_trial(agent_type, steps=6):
    env = VacuumEnvironment()
    model_agent = ModelBasedAgent() if agent_type == 'model' else None
    for _ in range(steps):
        percept = env.percept()
        if agent_type == 'reflex':
            action = simple_reflex_agent(percept)
        else:
            action = model_agent.act(percept)
        env.execute(action)
    return env.performance

random.seed(42)
reflex_score = run_trial('reflex')
random.seed(42)
model_score = run_trial('model')
print('Simple Reflex Agent performance:', reflex_score)
print('Model-Based Agent performance:', model_score)

```

Expected Output / Discussion

Both agents' cumulative performance scores are printed. Students should observe that the model-based agent avoids unnecessary movement once both locations are known to be clean, typically matching or exceeding the reflex agent's score while taking fewer wasted actions.

Learning Outcomes

- Understand and apply the PEAS framework to a concrete environment.
- Implement and differentiate simple reflex vs. model-based agents in code.
- Design and compute a performance measure for an agent.
- Connect the Week 1 agent taxonomy to working implementations.

Lab 3: Implement & Visualize BFS/DFS for a Maze Pathfinding Problem

Objective

Implement Breadth-First Search and Depth-First Search to solve a grid-maze pathfinding problem, and visualize the explored nodes and final path.

Background

Maze pathfinding is the canonical way to make uninformed search (Week 3) tangible: the maze is a graph where each open cell is a node and edges connect adjacent open cells. BFS explores level-by-level (guaranteeing the shortest path under unit costs); DFS plunges depth-first and may return a much longer path. Visualizing the search process makes the difference in exploration behavior concrete.

Lab Task

1. Represent a maze as a 2D grid (0 = open cell, 1 = wall), with a defined start and goal cell.
2. Implement BFS using a FIFO queue (`collections.deque`), tracking visited cells and parent pointers for path reconstruction.
3. Implement DFS using a stack (or recursion), with the same tracking mechanism.
4. Reconstruct and print the path found by each algorithm from start to goal.
5. Visualize the maze, the cells explored, and the final path using Matplotlib (e.g., different colors for walls, explored cells, and the path).

Code Example (Python)

```
from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0,0,0,1,0],
    [1,1,0,1,0],
    [0,0,0,0,0],
    [0,1,1,1,0],
    [0,0,0,0,0]
]
start, goal = (0, 0), (4, 4)

def neighbors(cell, maze):
    r, c = cell
    candidates = [(r-1,c), (r+1,c), (r,c-1), (r,c+1)]
    result = []
    for nr, nc in candidates:
        if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]) and maze[nr][nc] == 0:
            result.append((nr, nc))
    return result

def reconstruct_path(parent, start, goal):
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    path.reverse()
    return path

def bfs(maze, start, goal):
    frontier = deque([start])
    visited = {start}
    parent = {}
```

```

while frontier:
    current = frontier.popleft()
    if current == goal:
        return reconstruct_path(parent, start, goal), visited
    for nxt in neighbors(current, maze):
        if nxt not in visited:
            visited.add(nxt)
            parent[nxt] = current
            frontier.append(nxt)
    return None, visited

def dfs(maze, start, goal):
    frontier = [start]
    visited = {start}
    parent = {}
    while frontier:
        current = frontier.pop()
        if current == goal:
            return reconstruct_path(parent, start, goal), visited
        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                frontier.append(nxt)
    return None, visited

bfs_path, bfs_visited = bfs(maze, start, goal)
dfs_path, dfs_visited = dfs(maze, start, goal)
print('BFS path length:', len(bfs_path), '| cells explored:', len(bfs_visited))
print('DFS path length:', len(dfs_path), '| cells explored:', len(dfs_visited))

def visualize(maze, path, visited, title):
    grid = np.array(maze, dtype=float)
    for cell in visited:
        grid[cell] = 0.5
    for cell in path:
        grid[cell] = 0.8
    plt.imshow(grid, cmap='viridis')
    plt.title(title)
    plt.savefig(title.replace(' ', '_') + '.png')
    plt.close()

visualize(maze, bfs_path, bfs_visited, 'BFS Maze Result')
visualize(maze, dfs_path, dfs_visited, 'DFS Maze Result')

```

Expected Output / Discussion

Console output reports path length and number of cells explored for both algorithms (BFS typically finds the shortest path while exploring a level-balanced set of cells; DFS may find a longer path having explored a different, often less balanced, set of cells). Two saved images visualize walls, explored cells, and the final path for each algorithm.

Learning Outcomes

- Represent a grid-based search problem programmatically.
- Implement BFS and DFS with correct frontier/visited/parent bookkeeping.
- Reconstruct a solution path from search metadata.
- Visually interpret and compare search behavior.

Lab 4: Compare Performance (Time / Nodes Expanded) of Uninformed vs. Informed Search

Objective

Empirically measure and compare the efficiency of uninformed search (BFS) and informed search (A*) on the same maze problem, quantifying the benefit of heuristic guidance.

Background

Week 4-5 material argues that informed search should expand far fewer nodes than uninformed search on the same problem, because the heuristic focuses exploration toward the goal. This lab makes that claim measurable: the same maze from Lab 3 is solved twice, and the number of nodes expanded and wall-clock time are recorded and compared directly.

Lab Task

1. Reuse the maze and BFS implementation from Lab 3, but instrument it to count nodes expanded and measure execution time.
2. Implement A* search using the Manhattan distance heuristic between each cell and the goal.
3. Run both algorithms on the same maze (and, for a stronger comparison, on a larger randomly generated maze).
4. Tabulate path length, nodes expanded, and execution time for both algorithms.
5. Discuss why A* expands fewer nodes and whether path length differs between the two (it should not, since both are optimal under unit step costs on this maze).

Code Example (Python)

```
import heapq, time
from collections import deque

def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def bfs_instrumented(maze, start, goal):
    frontier = deque([start])
    visited = {start}
    parent = {}
    expanded = 0
    while frontier:
        current = frontier.popleft()
        expanded += 1
        if current == goal:
            path = [goal]
            while path[-1] != start:
                path.append(parent[path[-1]])
            return list(reversed(path)), expanded
        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                frontier.append(nxt)
    return None, expanded

def a_star(maze, start, goal):
    frontier = [(manhattan(start, goal), 0, start)]
    g_cost = {start: 0}
    parent = {}
    expanded = 0
    while frontier:
```

```

    f, g, current = heapq.heappop(frontier)
    expanded += 1
    if current == goal:
        path = [goal]
        while path[-1] != start:
            path.append(parent[path[-1]])
        return list(reversed(path)), expanded
    for nxt in neighbors(current, maze):
        new_g = g + 1
        if nxt not in g_cost or new_g < g_cost[nxt]:
            g_cost[nxt] = new_g
            f_score = new_g + manhattan(nxt, goal)
            heapq.heappush(frontier, (f_score, new_g, nxt))
            parent[nxt] = current
    return None, expanded

# (neighbors() reused from Lab 3)
start, goal = (0, 0), (4, 4)

t0 = time.perf_counter()
bfs_path, bfs_expanded = bfs_instrumented(maze, start, goal)
t1 = time.perf_counter()

t2 = time.perf_counter()
astar_path, astar_expanded = a_star(maze, start, goal)
t3 = time.perf_counter()

print(f'{"Algorithm":<10>{"PathLen":<10>{"Expanded":<10>{"Time(ms)":<10}'}')
print(f'{"BFS":<10}{len(bfs_path):<10}{bfs_expanded:<10}{(t1-t0)*1000:<10.3f}')
print(f'{"A*":<10}{len(astar_path):<10}{astar_expanded:<10}{(t3-t2)*1000:<10.3f}')

```

Expected Output / Discussion

A comparison table showing path length (equal for both, since the maze has uniform step costs), the number of nodes expanded (typically lower for A*), and execution time in milliseconds for each algorithm — quantitatively demonstrating the efficiency gain from using a heuristic.

Learning Outcomes

- Instrument search algorithms to collect performance metrics.
- Implement A* search with an admissible heuristic (Manhattan distance).
- Empirically compare uninformed and informed search efficiency.
- Interpret timing/node-count results in light of Week 4-5 theory.

Lab 5: Implement A* Search to Solve the 8-Puzzle Problem

Objective

Apply A* search to a harder, more realistic search problem — the 8-puzzle — using the Manhattan distance heuristic, reinforcing state-space formulation skills.

Background

The 8-puzzle (a 3x3 sliding tile puzzle) is a classic AI benchmark: states are tile arrangements, actions slide the blank tile up/down/left/right, and the goal is a specific ordered arrangement. Because the state space is much larger than the maze in Labs 3-4 (up to $9!/2 \approx 181,440$ reachable states), this lab demonstrates why informed search is essential in practice, not just a theoretical nicety.

Lab Task

1. Represent the 8-puzzle state as a tuple of 9 values (0 represents the blank tile).
2. Implement a function to generate valid successor states by sliding the blank tile.
3. Implement the Manhattan distance heuristic: for each tile, the sum of horizontal and vertical distance from its current position to its goal position.
4. Implement A* search using a priority queue (heapq), using state tuples as dictionary keys for visited/g-cost tracking.
5. Solve a given scrambled puzzle, print the sequence of moves, and report the number of nodes expanded and solution depth.

Code Example (Python)

```
import heapq

GOAL = (1,2,3,4,5,6,7,8,0)

def manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile == 0:
            continue
        goal_index = GOAL.index(tile)
        r1, c1 = divmod(i, 3)
        r2, c2 = divmod(goal_index, 3)
        distance += abs(r1-r2) + abs(c1-c2)
    return distance

def get_successors(state):
    successors = []
    blank = state.index(0)
    r, c = divmod(blank, 3)
    moves = [(-1,0,'Up'), (1,0,'Down'), (0,-1,'Left'), (0,1,'Right')]
    for dr, dc, name in moves:
        nr, nc = r+dr, c+dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_blank = nr*3 + nc
            new_state = list(state)
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
            successors.append((tuple(new_state), name))
    return successors

def a_star_puzzle(start):
    counter = 0 # tie-breaker for heapq comparisons
    frontier = [(manhattan_distance(start), 0, counter, start, [])]
    best_g = {start: 0}
```

```

expanded = 0
while frontier:
    f, g, _, state, path = heapq.heappop(frontier)
    expanded += 1
    if state == GOAL:
        return path, expanded
    for new_state, move in get_successors(state):
        new_g = g + 1
        if new_state not in best_g or new_g < best_g[new_state]:
            best_g[new_state] = new_g
            counter += 1
            h = manhattan_distance(new_state)
            heapq.heappush(frontier, (new_g+h, new_g, counter, new_state, path+[move]))
return None, expanded

start_state = (1,2,3,4,0,6,7,5,8) # a scrambled, solvable puzzle
solution, nodes_expanded = a_star_puzzle(start_state)
print('Solution moves:', solution)
print('Solution depth:', len(solution))
print('Nodes expanded:', nodes_expanded)

```

Expected Output / Discussion

The console prints the exact sequence of moves (e.g., ['Down', 'Right']) that transforms the scrambled start state into the goal state, plus the solution depth and total nodes expanded during A* search.

Learning Outcomes

- Formulate a real, non-trivial problem as a search problem (states, actions, goal test).
- Design and implement an admissible, consistent heuristic (Manhattan distance).
- Implement A* with a priority queue and proper tie-breaking.
- Appreciate the necessity of heuristic search for large state spaces.

Lab 6: Implement Hill Climbing; Analyze Its Tendency to Get Stuck in Local Optima

Objective

Implement steepest-ascent hill climbing for an optimization problem, and empirically demonstrate — and then partially remedy — its susceptibility to local optima.

Background

Week 6 covers hill climbing's core weakness: it can converge to a local (rather than global) optimum, especially on landscapes with many peaks. This lab uses the 8-Queens problem (minimize attacking pairs) as the test case, since it has many local optima, making the failure mode easy to observe and measure.

Lab Task

1. Represent an 8-Queens state as a list of 8 integers (row of the queen in each column).
2. Implement an objective function counting the number of attacking pairs of queens (to be minimized).
3. Implement steepest-ascent hill climbing: at each step, examine all single-queen-move neighbors and move to the best one; stop when no neighbor improves the objective.
4. Run hill climbing from many random initial states (e.g., 100 runs) and record how often it reaches a true solution (0 attacking pairs) versus getting stuck at a local optimum (>0 attacking pairs, no improving neighbor).
5. Implement random-restart hill climbing (repeat step 3 with a fresh random state whenever stuck) and compare its success rate to plain hill climbing.

Code Example (Python)

```
import random

N = 8

def attacking_pairs(state):
    count = 0
    for i in range(N):
        for j in range(i+1, N):
            if state[i] == state[j] or abs(state[i]-state[j]) == abs(i-j):
                count += 1
    return count

def get_neighbors(state):
    neighbors = []
    for col in range(N):
        for row in range(N):
            if row != state[col]:
                new_state = list(state)
                new_state[col] = row
                neighbors.append(new_state)
    return neighbors

def hill_climb(state):
    current = state
    current_cost = attacking_pairs(current)
    while True:
        neighbors = get_neighbors(current)
        best_neighbor = min(neighbors, key=attacking_pairs)
        best_cost = attacking_pairs(best_neighbor)
        if best_cost >= current_cost:
            return current, current_cost # local optimum (or solution) reached
```

```

        current, current_cost = best_neighbor, best_cost

def random_state():
    return [random.randint(0, N-1) for _ in range(N)]

def random_restart_hill_climb(max_restarts=50):
    for _ in range(max_restarts):
        result, cost = hill_climb(random_state())
        if cost == 0:
            return result, cost
    return result, cost # return last attempt if none succeeded

random.seed(1)
successes, stuck = 0, 0
for _ in range(100):
    _, cost = hill_climb(random_state())
    if cost == 0:
        successes += 1
    else:
        stuck += 1
print(f'Plain hill climbing: {successes}/100 reached a true solution, {stuck}/100 stuck in a local optimum')

restart_successes = sum(1 for _ in range(20) if random_restart_hill_climb()[1] == 0)
print(f'Random-restart hill climbing: {restart_successes}/20 runs succeeded')

```

Expected Output / Discussion

Plain hill climbing typically solves roughly 14-16% of random 8-Queens starts (the rest get stuck at local optima with 1-2 attacking pairs remaining), while random-restart hill climbing succeeds on nearly all runs (since it keeps retrying from new starting points until it finds a global optimum).

Learning Outcomes

- Implement steepest-ascent hill climbing correctly.
- Design and use an objective function for an optimization problem.
- Empirically observe and quantify the local-optima failure mode.
- Implement and evaluate random-restart hill climbing as a mitigation.

Lab 7: Develop a Tic-Tac-Toe Game with the Min-Max Algorithm

Objective

Build a fully playable Tic-Tac-Toe game where the computer opponent plays optimally using the Min-Max algorithm, reinforcing adversarial search from Week 7.

Background

Tic-Tac-Toe has a small enough game tree (at most $9! = 362,880$ nodes, far fewer in practice) to make full minimax tractable, making it the standard first adversarial-search project. This lab implements the complete game loop, terminal-state detection, and a minimax function that recursively evaluates the game tree to select the optimal move for the AI player.

Lab Task

1. Represent the board as a list of 9 cells ('X', 'O', or empty).
2. Implement a function to check for a winner or a draw (terminal test + utility).
3. Implement the minimax function: MAX (the AI, e.g., 'X') picks the move maximizing the outcome; MIN (the opponent, 'O') picks the move minimizing it; utility values are +1 (AI win), -1 (AI loss), 0 (draw).
4. Implement a `best_move()` function that evaluates all legal moves via minimax and returns the optimal one.
5. Build a simple game loop where a human plays against the AI, and verify the AI never loses (it will win or draw every game).

Code Example (Python)

```
def check_winner(board):
    lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,b,c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    if ' ' not in board:
        return 'Draw'
    return None

def minimax(board, is_maximizing):
    result = check_winner(board)
    if result == 'X':
        return 1
    if result == 'O':
        return -1
    if result == 'Draw':
        return 0

    if is_maximizing:
        best_score = float('-inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(board, False)
                board[i] = ' '
                best_score = max(best_score, score)
        return best_score
    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(board, True)
                board[i] = ' '
```

```

        best_score = min(best_score, score)
    return best_score

def best_move(board):
    best_score = float('-inf')
    move = None
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    return move

def print_board(board):
    for r in range(0, 9, 3):
        print('|'.join(board[r:r+3]))

# Simple demonstration: AI ('X') moves first, always optimally
board = [' '] * 9
board[4] = 'O' # opponent has already taken the center
ai_move = best_move(board)
board[ai_move] = 'X'
print('AI chose position:', ai_move)
print_board(board)

```

Expected Output / Discussion

The AI's chosen move is printed along with the resulting board — the AI will select a corner (the theoretically correct optimal response when the opponent has taken the center), demonstrating that minimax reliably finds game-theoretically optimal play.

Learning Outcomes

- Implement terminal-state detection and a utility function for a two-player game.
- Implement the recursive minimax algorithm correctly, alternating MAX/MIN layers.
- Use minimax to drive optimal move selection in a real, playable game.
- Verify empirically that minimax play never loses.

Lab 8: Enhance the Game with Alpha-Beta Pruning and Compare Efficiency

Objective

Extend the Lab 7 Tic-Tac-Toe AI with alpha-beta pruning, and empirically measure the reduction in nodes evaluated compared to plain minimax — reinforcing Week 8's pruning material.

Background

Alpha-beta pruning computes the exact same minimax value while skipping provably irrelevant branches. On Tic-Tac-Toe's small tree the speedup is modest but measurable; the lab's real value is having students trace, in code, precisely where and why branches get pruned, and see the node-count reduction as concrete evidence rather than only a theoretical claim.

Lab Task

1. Modify the Lab 7 minimax function to accept and propagate alpha and beta bounds.
2. Add a pruning condition: break out of the loop when $\alpha \geq \beta$.
3. Instrument both the plain minimax and the alpha-beta version to count the number of board states evaluated.
4. Run `best_move()` using both versions on the same empty (or partially filled) board and compare node counts.
5. Discuss how move ordering (e.g., trying the center and corners before edges) affects the amount of pruning achieved.

Code Example (Python)

```
node_count = {'plain': 0, 'pruned': 0}

def minimax_plain(board, is_maximizing):
    node_count['plain'] += 1
    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0
    scores = []
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X' if is_maximizing else 'O'
            scores.append(minimax_plain(board, not is_maximizing))
            board[i] = ' '
    return max(scores) if is_maximizing else min(scores)

def minimax_alphabeta(board, is_maximizing, alpha, beta):
    node_count['pruned'] += 1
    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0

    if is_maximizing:
        best_score = float('-inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax_alphabeta(board, False, alpha, beta)
                board[i] = ' '
                best_score = max(best_score, score)
                alpha = max(alpha, best_score)
                if alpha >= beta:
                    break # prune: MIN will never let play reach this branch
        return best_score
    else:
```

```

    best_score = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax_alphabeta(board, True, alpha, beta)
            board[i] = ' '
            best_score = min(best_score, score)
            beta = min(beta, best_score)
            if alpha >= beta:
                break # prune: MAX will never let play reach this branch
    return best_score

board = [' '] * 9
minimax_plain(list(board), True)
minimax_alphabeta(list(board), True, float('-inf'), float('inf'))

print('Nodes evaluated (plain minimax):', node_count['plain'])
print('Nodes evaluated (alpha-beta):', node_count['pruned'])
reduction = 100 * (1 - node_count['pruned'] / node_count['plain'])
print(f'Reduction: {reduction:.1f}%')

```

Expected Output / Discussion

Two node counts are printed for evaluating the full game tree from an empty board — alpha-beta pruning evaluates substantially fewer nodes than plain minimax (on the order of tens of percent fewer, depending on move ordering), with the percentage reduction printed directly.

Learning Outcomes

- Correctly implement alpha-beta pruning as an extension of minimax.
- Instrument code to quantitatively measure algorithmic efficiency.
- Empirically confirm that pruning preserves the minimax result while reducing computation.
- Reason about how move ordering affects pruning effectiveness.

Lab 9: Build a Small Rule-Based Expert System (Medical Diagnosis Example)

Objective

Design and implement a simple forward-chaining rule-based expert system that infers a diagnosis from a set of observed symptoms, connecting to the knowledge-representation and inference material from Week 2.

Background

Expert systems were among AI's earliest commercial successes (e.g., MYCIN, covered in Week 1's history). They encode domain expert knowledge as IF-THEN rules and use an inference engine to chain facts through rules until a conclusion is reached (forward chaining) or to work backward from a hypothesis to check supporting evidence (backward chaining). This lab builds a small forward-chaining engine for a toy medical-diagnosis knowledge base.

Lab Task

1. Define a knowledge base of IF-THEN rules, each with a set of required symptom facts and a resulting diagnosis fact (e.g., IF fever AND cough AND sore_throat THEN flu).
2. Represent the current set of known facts as a Python set, initialized with the patient's reported symptoms.
3. Implement a forward-chaining inference engine: repeatedly scan all rules, and whenever a rule's conditions are all satisfied by current facts, add its conclusion to the facts (if not already present).
4. Continue until no new facts can be inferred in a full pass over all rules (fixed-point reached).
5. Test the system with different sets of initial symptoms and confirm it reaches the medically 'expected' diagnosis, and discuss the system's limitations (no uncertainty handling, brittle exact-match rules).

Code Example (Python)

```
# Each rule: (frozenset of required facts, resulting fact)
rules = [
    ({'fever', 'cough', 'sore_throat'}, 'flu'),
    ({'fever', 'rash', 'joint_pain'}, 'dengue'),
    ({'flu', 'chest_pain'}, 'possible_pneumonia'),
    ({'sneezing', 'runny_nose', 'itchy_eyes'}, 'allergy'),
    ({'possible_pneumonia'}, 'refer_to_specialist'),
]

def forward_chain(facts, rules):
    facts = set(facts)
    changed = True
    fired_rules = []
    while changed:
        changed = False
        for conditions, conclusion in rules:
            if conditions.issubset(facts) and conclusion not in facts:
                facts.add(conclusion)
                fired_rules.append((conditions, conclusion))
                changed = True
    return facts, fired_rules

patient_symptoms = {'fever', 'cough', 'sore_throat', 'chest_pain'}
final_facts, fired = forward_chain(patient_symptoms, rules)

print('Initial symptoms:', patient_symptoms)
print('All inferred facts:', final_facts)
print()
print('Rules fired (in order):')
for conditions, conclusion in fired:
    print(f' IF {conditions} THEN {conclusion}')
```

Expected Output / Discussion

The system prints the initial symptoms, then chains through the rules: flu is inferred first, then possible_pneumonia (since flu + chest_pain match a rule), then refer_to_specialist — demonstrating multi-step forward-chaining inference from raw facts to a final recommendation.

Learning Outcomes

- Represent domain knowledge as IF-THEN production rules.
- Implement a working forward-chaining inference engine.
- Trace multi-step chains of inference through a rule base.
- Critically assess the limitations of purely symbolic, rule-based reasoning.

Lab 10: Build a Decision Tree Classifier From Scratch

Objective

Implement the ID3 decision-tree-building algorithm from scratch (entropy and information gain, recursive splitting) on a small classification dataset, directly operationalizing the Week 10 mathematics.

Background

Understanding decision trees deeply requires implementing entropy and information gain by hand at least once, rather than only calling a library. This lab builds the classic 'Play Tennis' decision tree using categorical features, recursively selecting the highest-information-gain attribute at each node, exactly following the ID3 procedure covered in lecture.

Lab Task

1. Represent the dataset as a list of dictionaries (rows), each with feature values and a target label.
2. Implement an `entropy()` function operating on a list of labels.
3. Implement an `information_gain()` function that computes the entropy reduction from splitting the dataset on a given attribute.
4. Implement a recursive `build_tree()` function that selects the attribute with highest information gain, splits the data, and recurses on each subset (stopping when a subset is pure or attributes are exhausted).
5. Print the resulting tree structure and use it to classify a new, unseen example.

Code Example (Python)

```
import math
from collections import Counter

data = [
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'Yes'},
    {'Outlook': 'Sunny', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
]

def entropy(rows):
    counts = Counter(row['Play'] for row in rows)
    total = len(rows)
    return -sum((c/total) * math.log2(c/total) for c in counts.values())

def information_gain(rows, attribute):
    total_entropy = entropy(rows)
    values = set(row[attribute] for row in rows)
    weighted_entropy = 0
    for v in values:
        subset = [row for row in rows if row[attribute] == v]
        weighted_entropy += (len(subset)/len(rows)) * entropy(subset)
    return total_entropy - weighted_entropy

def build_tree(rows, attributes):
    labels = [row['Play'] for row in rows]
    if labels.count(labels[0]) == len(labels):
        return labels[0] # pure node
    if not attributes:
        return Counter(labels).most_common(1)[0][0] # majority vote
```

```

gains = {attr: information_gain(rows, attr) for attr in attributes}
best_attr = max(gains, key=gains.get)

tree = {best_attr: {}}
remaining_attrs = [a for a in attributes if a != best_attr]
for value in set(row[best_attr] for row in rows):
    subset = [row for row in rows if row[best_attr] == value]
    tree[best_attr][value] = build_tree(subset, remaining_attrs)
return tree

def classify(tree, example):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = example[attribute]
    subtree = tree[attribute].get(value, 'Unknown')
    return classify(subtree, example)

attributes = ['Outlook', 'Temp', 'Humidity', 'Wind']
tree = build_tree(data, attributes)
print('Decision tree:', tree)

new_example = {'Outlook': 'Sunny', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong'}
print('Prediction for new example:', classify(tree, new_example))

```

Expected Output / Discussion

The printed tree structure shows 'Outlook' as the root split (matching the by-hand calculation from lecture, since it has the highest information gain), with nested sub-trees for the Rain branch. The new example is classified using the learned tree.

Learning Outcomes

- Implement entropy and information gain from their mathematical definitions.
- Implement the recursive ID3 tree-construction algorithm.
- Represent and traverse a learned decision tree programmatically.
- Use a from-scratch model to classify unseen data.

Lab 11: Implement a Naïve Bayes Classifier for a Spam Detection Task

Objective

Implement a Multinomial Naïve Bayes text classifier from first principles to classify short messages as spam or not spam, applying Week 12's Bayes'-Theorem-based classification.

Background

Spam filtering is the textbook Naïve Bayes application: each word in a message is treated as a conditionally-independent feature given the class (spam/not spam), and Bayes' Theorem combines word-level likelihoods with class priors to compute the most probable class. This lab implements the full pipeline — tokenization, probability estimation with Laplace smoothing, and classification — without relying on a library.

Lab Task

1. Build a small labeled training set of short messages, each tagged 'spam' or 'ham' (not spam).
2. Tokenize messages into lowercase words and build a vocabulary.
3. Compute class priors $P(\text{spam})$ and $P(\text{ham})$, and word likelihoods $P(\text{word}|\text{class})$ using Laplace (add-1) smoothing to avoid zero probabilities.
4. Implement a `classify()` function that computes the (log) posterior probability for each class given a new message's words, and returns the higher-probability class.
5. Test the classifier on a few new messages and discuss the effect of the independence assumption and smoothing on the results.

Code Example (Python)

```
import math
from collections import Counter, defaultdict

training_data = [
    ('win free money now', 'spam'),
    ('claim your free prize today', 'spam'),
    ('limited offer act now', 'spam'),
    ('meeting scheduled for tomorrow', 'ham'),
    ('can we discuss the project plan', 'ham'),
    ('lunch at noon works for me', 'ham'),
]

def tokenize(text):
    return text.lower().split()

class_word_counts = defaultdict(Counter)
class_totals = Counter()
vocabulary = set()
doc_counts = Counter()

for text, label in training_data:
    words = tokenize(text)
    doc_counts[label] += 1
    for w in words:
        class_word_counts[label][w] += 1
        class_totals[label] += 1
        vocabulary.add(w)

total_docs = sum(doc_counts.values())
vocab_size = len(vocabulary)
```

```

def class_prior(label):
    return doc_counts[label] / total_docs

def word_likelihood(word, label):
    # Laplace (add-1) smoothing
    count = class_word_counts[label][word]
    return (count + 1) / (class_totals[label] + vocab_size)

def classify(text):
    words = tokenize(text)
    scores = {}
    for label in doc_counts:
        log_prob = math.log(class_prior(label))
        for w in words:
            log_prob += math.log(word_likelihood(w, label))
        scores[label] = log_prob
    return max(scores, key=scores.get), scores

test_messages = [
    'win a free trip now',
    'are we still on for the meeting',
]

for msg in test_messages:
    prediction, scores = classify(msg)
    print(f'Message: "{msg}"')
    print(f' Predicted class: {prediction} (log-scores: {scores})')

```

Expected Output / Discussion

For each test message, the classifier prints the predicted class (spam or ham) along with the raw log-probability scores for both classes, showing how word-level evidence combines with the class prior to reach a decision.

Learning Outcomes

- Implement class priors and smoothed word likelihoods from raw counts.
- Apply Bayes' Theorem (in log-space, to avoid numerical underflow) for text classification.
- Understand the practical role of Laplace smoothing.
- Critically evaluate the conditional-independence assumption underlying Naïve Bayes.

Lab 12: Use scikit-learn to Compare Naïve Bayes and SVM on a Dataset

Objective

Use scikit-learn to train and evaluate both a Naïve Bayes and an SVM classifier on the same dataset, and write a short comparison report of their performance and characteristics.

Background

Having built Naïve Bayes from scratch in Lab 11, this lab shifts to industry-standard tooling, comparing it against Support Vector Machines (Week 12) using scikit-learn's consistent fit/predict API. The goal is both a practical skill (using a production-grade ML library) and an empirical grounding for the theoretical Naïve-Bayes-vs-SVM comparison discussed in lecture.

Lab Task

1. Load a classification dataset (e.g., the built-in scikit-learn 'Iris' dataset, or a small text dataset such as 20-Newsgroups subset for a closer spam-style comparison).
2. Split the data into training and test sets (e.g., 80/20 split) using `train_test_split`.
3. Train a GaussianNB (or MultinomialNB for text) classifier and an SVC (Support Vector Classifier) on the training set.
4. Evaluate both models on the test set using accuracy, precision, recall, and F1-score (`classification_report`).
5. Write a short comparison report (a few paragraphs) covering: which model performed better and why, training time differences, and how each model's assumptions (feature independence for NB; margin maximization for SVM) relate to the observed results.

Code Example (Python)

```
import time
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score

# 1. Load data
data = load_iris()
X, y = data.data, data.target

# 2. Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Train Naive Bayes
t0 = time.perf_counter()
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_train_time = time.perf_counter() - t0
nb_predictions = nb_model.predict(X_test)

# 4. Train SVM
t0 = time.perf_counter()
svm_model = SVC(kernel='rbf', C=1.0)
svm_model.fit(X_train, y_train)
svm_train_time = time.perf_counter() - t0
svm_predictions = svm_model.predict(X_test)

# 5. Evaluate and compare
print('--- Naive Bayes ---')
```

```
print('Accuracy:', accuracy_score(y_test, nb_predictions))
print('Training time (s):', round(nb_train_time, 5))
print(classification_report(y_test, nb_predictions, target_names=data.target_names))

print('--- SVM (RBF kernel) ---')
print('Accuracy:', accuracy_score(y_test, svm_predictions))
print('Training time (s):', round(svm_train_time, 5))
print(classification_report(y_test, svm_predictions, target_names=data.target_names))
```

Expected Output / Discussion

Two full classification reports (precision, recall, F1-score per class, plus overall accuracy) are printed, along with training time for each model — typically both models achieve high accuracy (>90%) on Iris, with SVM often slightly ahead on the harder-to-separate classes, and Naive Bayes training noticeably faster.

Learning Outcomes

- Use scikit-learn's standard train/predict/evaluate workflow.
- Correctly apply and interpret classification_report metrics.
- Empirically compare two classification algorithms on identical data splits.
- Write a concise, evidence-based technical comparison report.

Lab 13: Build and Train a Basic Feedforward Network for Digit Classification

Objective

Use TensorFlow/Keras (or PyTorch) to build, train, and evaluate a simple feedforward neural network that classifies handwritten digits (MNIST), applying the Week 13-14 neural network material at production-library scale.

Background

Having implemented a perceptron and backpropagation by hand in Lab 14 (XOR), this lab shows the same underlying concepts (weighted sums, activation functions, gradient-based training) operating at real scale on the MNIST handwritten-digit dataset (70,000 28x28 grayscale images), using a modern deep learning framework that handles the calculus automatically.

Lab Task

1. Load the MNIST dataset (available directly via `tf.keras.datasets.mnist`).
2. Preprocess the data: flatten each 28x28 image into a 784-length vector and normalize pixel values to [0, 1].
3. Build a Sequential feedforward network: an input layer, one or two hidden Dense layers with ReLU activation, and an output Dense layer with 10 units and softmax activation.
4. Compile the model with an appropriate loss (`sparse_categorical_crossentropy`), optimizer (`adam`), and metric (`accuracy`), then train for several epochs.
5. Evaluate the trained model on the held-out test set and plot the training/validation accuracy curves across epochs.

Code Example (Python)

```
import tensorflow as tf
import matplotlib.pyplot as plt

# 1. Load MNIST
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# 2. Preprocess: flatten and normalize
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test = x_test.reshape(-1, 784).astype('float32') / 255.0

# 3. Build the feedforward network
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

# 4. Compile and train
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=10, validation_split=0.1, batch_size=32)

# 5. Evaluate and visualize
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print('Test accuracy:', test_accuracy)

plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='validation accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.title('MNIST Feedforward Network Training')  
plt.savefig('lab13_training_curve.png')
```

Expected Output / Discussion

Training progress is printed epoch by epoch (loss/accuracy for both training and validation), the final test accuracy is printed (typically around 97-98% for this architecture), and a saved plot shows training vs. validation accuracy converging over epochs.

Learning Outcomes

- Build a multi-layer feedforward network using a modern deep learning framework.
- Correctly preprocess image data for a dense network (flattening, normalization).
- Configure an appropriate loss function, optimizer, and evaluation metric.
- Interpret training/validation curves to assess model fit.

Lab 14: Implement an MLP From Scratch to Solve XOR, Demonstrating Backpropagation

Objective

Implement a Multi-Layer Perceptron and the backpropagation algorithm entirely from scratch using only NumPy, training it to solve the XOR problem — directly operationalizing the Week 13-14 mathematics without any deep learning framework.

Background

As covered in Week 13, a single-layer perceptron structurally cannot learn XOR (it is not linearly separable). This lab proves that an MLP with one hidden layer, trained via backpropagation, can — implementing every step (forward pass, loss computation, backward pass via the chain rule, weight updates) manually, so the mechanics of Week 14's lecture become tangible code.

Lab Task

1. Define the XOR truth table as the training data (4 examples).
2. Initialize random weights and biases for a network with 2 input units, a hidden layer (e.g., 2-4 units) with sigmoid activation, and 1 output unit with sigmoid activation.
3. Implement the forward pass: compute hidden-layer and output-layer activations.
4. Implement the backward pass: compute the output error, propagate it back through the hidden layer using the chain rule and the sigmoid derivative, and compute gradients for all weights and biases.
5. Train for many epochs using gradient descent, then verify the trained network correctly predicts all four XOR outputs (something a single-layer perceptron could never achieve).

Code Example (Python)

```
import numpy as np

np.random.seed(1)

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]]) # XOR targets

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

input_size, hidden_size, output_size = 2, 4, 1
learning_rate = 0.5

W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
b2 = np.zeros((1, output_size))

for epoch in range(10000):
    # Forward pass
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    # Loss (mean squared error)
    error = y - final_output
```

```

# Backward pass (chain rule)
d_output = error * sigmoid_derivative(final_output)
error_hidden = d_output.dot(W2.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# Weight/bias updates (gradient descent)
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
W1 += X.T.dot(d_hidden) * learning_rate
b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

if epoch % 2000 == 0:
    loss = np.mean(np.square(error))
    print(f'Epoch {epoch}, Loss: {loss:.5f}')

print()
print('Final predictions after training:')
for i in range(4):
    print(f'Input: {X[i]} -> Predicted: {final_output[i][0]:.4f}, Target: {y[i][0]}')

```

Expected Output / Discussion

Loss decreases over training epochs (printed periodically), and the final predictions closely match the true XOR outputs (e.g., predictions near 0.02, 0.98, 0.98, 0.02 for targets 0, 1, 1, 0) — direct empirical proof that the hidden layer enables learning a non-linearly-separable function.

Learning Outcomes

- Implement forward propagation through a multi-layer network by hand.
- Implement backpropagation via the chain rule, without any library assistance.
- Implement gradient-descent-based weight updates.
- Empirically confirm that an MLP overcomes the single-layer perceptron's XOR limitation.

Lab 15: Implement a Genetic Algorithm to Solve an Optimization Problem (Eight Queens)

Objective

Implement a complete Genetic Algorithm — encoding, fitness function, selection, crossover, and mutation — to solve the 8-Queens problem, applying Week 15's evolutionary computation material to a familiar optimization problem (also used in Lab 6, enabling direct comparison with hill climbing).

Background

Because Lab 6 already explored 8-Queens with hill climbing (and observed its local-optima problem), using the same problem here lets students directly compare a population-based, stochastic global search method (GA) against a single-state local search method, reinforcing the Week 15 comparison table from lecture.

Lab Task

1. Encode an individual as a list of 8 integers (row of the queen in each column), matching Lab 6's representation.
2. Implement a fitness function: since GAs conventionally maximize fitness, use $(28 - \text{attacking_pairs})$, since 28 is the maximum possible number of pairs among 8 queens (0 attacking pairs = fitness 28 = a solution).
3. Implement selection (e.g., tournament selection), single-point crossover, and mutation (randomly reassign one queen's row with some probability).
4. Implement the main GA loop: initialize a random population, then repeatedly select parents, apply crossover and mutation to produce the next generation, until a solution (fitness = 28) is found or a maximum generation count is reached.
5. Run the GA multiple times and record the generation at which a solution is found; compare against Lab 6's hill-climbing success rate.

Code Example (Python)

```
import random

N = 8
POPULATION_SIZE = 100
MUTATION_RATE = 0.1
MAX_GENERATIONS = 1000
MAX_FITNESS = 28 # C(8,2) = 28 total pairs; 28 - attacking_pairs = fitness

def random_individual():
    return [random.randint(0, N-1) for _ in range(N)]

def fitness(individual):
    attacking = 0
    for i in range(N):
        for j in range(i+1, N):
            if individual[i] == individual[j] or abs(individual[i]-individual[j]) == abs(i-j):
                attacking += 1
    return MAX_FITNESS - attacking

def tournament_selection(population, k=5):
    contenders = random.sample(population, k)
    return max(contenders, key=fitness)

def crossover(parent1, parent2):
    point = random.randint(1, N-1)
    child = parent1[:point] + parent2[point:]
    return child

def mutate(individual):
```

```

    if random.random() < MUTATION_RATE:
        col = random.randint(0, N-1)
        individual[col] = random.randint(0, N-1)
    return individual

def genetic_algorithm():
    population = [random_individual() for _ in range(POPULATION_SIZE)]
    for generation in range(MAX_GENERATIONS):
        population.sort(key=fitness, reverse=True)
        if fitness(population[0]) == MAX_FITNESS:
            return population[0], generation

        new_population = population[:5] # elitism: keep top 5
        while len(new_population) < POPULATION_SIZE:
            parent1 = tournament_selection(population)
            parent2 = tournament_selection(population)
            child = crossover(parent1, parent2)
            child = mutate(child)
            new_population.append(child)
        population = new_population
    return population[0], MAX_GENERATIONS

random.seed(7)
solution, generation_found = genetic_algorithm()
print('Best solution:', solution)
print('Fitness:', fitness(solution), '/', MAX_FITNESS)
print('Solution found at generation:', generation_found)

```

Expected Output / Discussion

The GA typically converges to a perfect solution (fitness 28, zero attacking pairs) within a few dozen to a few hundred generations, printed along with the exact board arrangement and the generation number at which it was found — illustrating how population diversity and crossover help escape the local optima that trap plain hill climbing.

Learning Outcomes

- Implement all core GA components: encoding, fitness, selection, crossover, mutation.
- Implement a complete generational GA loop with elitism.
- Apply a GA to a concrete optimization problem already studied under a different method.
- Compare population-based stochastic search against single-state local search empirically.

Lab 16: Develop a Small-Scale AI Application: Image Classifier Using a Pre-Trained CNN

Objective

Build a practical, end-to-end image classification application using transfer learning with a pre-trained Convolutional Neural Network, synthesizing the Week 16 CNN material into a deployable mini-application and capstone project.

Background

Training a CNN from scratch on a small dataset rarely works well; the standard real-world practice is transfer learning — taking a network pre-trained on a large dataset (e.g., ImageNet) and either using it directly or fine-tuning its final layers for a new, smaller task. This capstone lab uses a pre-trained MobileNetV2 model to build a simple, working image classifier, tying together the CNN concepts from Week 16 with a genuinely useful application.

Lab Task

1. Load a pre-trained MobileNetV2 model (trained on ImageNet) via `tf.keras.applications`.
2. Write a function to load and preprocess an input image to the format MobileNetV2 expects (resizing to 224x224, appropriate pixel scaling).
3. Run the pre-trained model directly for general-purpose classification (zero-shot, using the full 1000 ImageNet classes) and decode the top predictions.
4. (Extension) Freeze the pre-trained convolutional base and add a new small classification head, then fine-tune on a small custom dataset (e.g., a 2-3 class flower or cats-vs-dogs subset) to specialize the model for a new task.
5. Wrap the model in a simple `predict_image(path)` function that returns the top-3 predicted labels with confidence scores, forming a minimal but complete AI application.

Code Example (Python)

```
import tensorflow as tf
import numpy as np

# 1. Load the pre-trained model
model = tf.keras.applications.MobileNetV2(weights='imagenet')

# 2. Preprocessing function
def load_and_preprocess(image_path):
    img = tf.keras.utils.load_img(image_path, target_size=(224, 224))
    img_array = tf.keras.utils.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    return tf.keras.applications.mobilenet_v2.preprocess_input(img_array)

# 3. Zero-shot classification function
def predict_image(image_path, top_k=3):
    processed = load_and_preprocess(image_path)
    predictions = model.predict(processed)
    decoded = tf.keras.applications.mobilenet_v2.decode_predictions(predictions, top=top_k)[0]
    results = [(label, float(confidence)) for (_, label, confidence) in decoded]
    return results

# Example usage
results = predict_image('sample_image.jpg')
print('Top predictions:')
for label, confidence in results:
    print(f' {label}: {confidence*100:.2f}%')

# 4. (Extension) Building a custom classification head for fine-tuning
base_model = tf.keras.applications.MobileNetV2(
```

```

        weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False    # freeze the pre-trained convolutional base

custom_model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')    # e.g., 3 custom classes
])
custom_model.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])
print(custom_model.summary())

```

Expected Output / Discussion

For the zero-shot classifier, the top-3 predicted ImageNet labels and confidence percentages are printed for a sample image (e.g., 'golden_retriever: 87.32%'). The extension section prints a model summary showing the frozen MobileNetV2 base plus a new trainable classification head, ready for fine-tuning on a custom dataset.

Learning Outcomes

- Apply transfer learning using a pre-trained CNN for practical image classification.
- Correctly preprocess images to match a pre-trained model's expected input format.
- Build and compile a custom classification head on top of a frozen convolutional base.
- Deliver a complete, working small-scale AI application, synthesizing the course's neural network content.